

Chapter 6

Randomization II

This recitation covers a number of examples of analyzing randomized algorithms, with a focus on high-probability bounds for their span.

1 High Probability Bounds

We say that an algorithm terminates in $O(f(n))$ **with high probability** if, for any value of $k > 0$, the algorithm terminates in $O(k \cdot f(n))$ time with probability at least $1 - \frac{1}{n^k}$.

2 Another Dice Game

Having won the previous dice game of Lucky 7s, Anna has decided to play another game with a board and a die. She starts with a number n on the board. On each round, Anna rolls the die to generate a random integer d between 1 and 6. She then multiplies the number on the board by $\frac{d}{4}$, and rounds it down. The game ends when Anna's number on the board is less than or equal to 1.

Task. Show that Anna's game ends in $O(\log n)$ rounds with high probability.

3 The Secret Algorithm

Task. Umut has a secret algorithm which has $O(\log n)$ span with high probability, and $O(n)$ span in the worst case. Prove that Umut's algorithm has $O(\log n)$ span in expectation.

Task. Guy is trying to guess what Umut's secret algorithm is. He comes up with the following algorithm:

```
fun rec(n : int) =  
  if n <= 1 then 1 else  
  case flip()  
  of true => rec(n/2) + 1  
   | false => rec(n-1) + 1
```

The flip function simulates a coin flip and returns true and false with equal probability. Is this a viable candidate for Umut's secret algorithm?

4 Parallel Randomization

We have a function f , which takes an input of size n . f is a randomized function, which has a $\frac{1}{2}$ chance of halving the size of the input on each round and a $\frac{1}{2}$ chance of leaving it the same size. f terminates when the input size is ≤ 1 , as always. From what we've done so far, we know that a single call to f will terminate in $O(\log n)$ rounds with high probability.

Daniel is writing a brute-force algorithm, and he is using f on each of the different possible solutions. In particular, his problem has n^{210} possible solutions, and he calls f on each of these n^{210} inputs of size n in parallel. His computer allows for one round of all running calls to f to take place each second – that is, after one second, all calls to f will have run one round, after two seconds, they will each have run two rounds, and so on.

Task. Prove that the number of seconds it takes for all the parallel calls to terminate is still in $O(\log n)$ with high probability.